

Executive Summary

Cube ACR is a dual-app system: **Cube ACR (main app)** + **Cube ACR Helper (connector app)**. The main app provides the UI, file management, cloud backup, and in-app playback; the helper runs invisibly (as an AccessibilityService and background service) to detect calls and capture audio. Together they enable two-way call recording even on Android 9–14, using Android's voice-recognition audio channel instead of speaker-based capture ¹. The apps communicate via Android IPC (broadcast intents and shared storage). Each app has its own permissions: for example, **Helper** holds RECORD_AUDIO, PROCESS_OUTGOING_CALLS, BIND_ACCESSIBILITY_SERVICE, etc., while **Main** holds READ_CONTACTS, SYSTEM_ALERT_WINDOW, FOREGROUND_SERVICE, etc. ² ³. On incoming/outgoing calls, the Helper detects the call state and starts recording; it saves the file and notifies the Main app, which then updates its database and UI. Permissions and special app accesses (Accessibility, Overlay, Battery-optimization ignore) must be granted by the user at runtime. The architecture follows best practices by isolating the recording engine in the helper and keeping the main app's privileges limited. We include example Android manifest snippets and code (e.g. using `PackageManager` to detect the helper, and `MediaRecorder/AudioRecord` to start recording) and Mermaid diagrams for call-event flow and IPC interactions.

System Architecture

Cube ACR is split into two cooperating apps:

- **Cube ACR (main app)** – Handles user interface, settings, storing recordings, and playback. It saves recordings (to a folder on external storage) and maintains a database/metadata.
- **Cube ACR Helper (connector app)** – Runs in background (as an AccessibilityService + foreground service). It listens for call events and performs the actual audio capture from both sides of the call.

This separation arose because Android 9+ forbids call recording via ordinary APIs; the Helper uses an accessibility-based workaround. The main app detects whether the Helper is installed (via `PackageManager`) and, if not, prompts the user to install it ⁴. Both apps declare distinct Android components and permissions in their manifests:

- *Main app components:* Activities (UI screens), a `Service` or `BroadcastReceiver` for call events (with `READ_PHONE_STATE`), and possibly a foreground service for ongoing recording. It likely uses a local `BroadcastReceiver` or bound service to interact with the helper.
- *Helper app components:* An **AccessibilityService** (declared with `<service android:permission="android.permission.BIND_ACCESSIBILITY_SERVICE">`), a background `Service` (marked foreground when recording), and `BroadcastReceiver` (e.g. for `BOOT_COMPLETED`). The AccessibilityService can receive global call-screen notifications, and the Service handles audio capture.

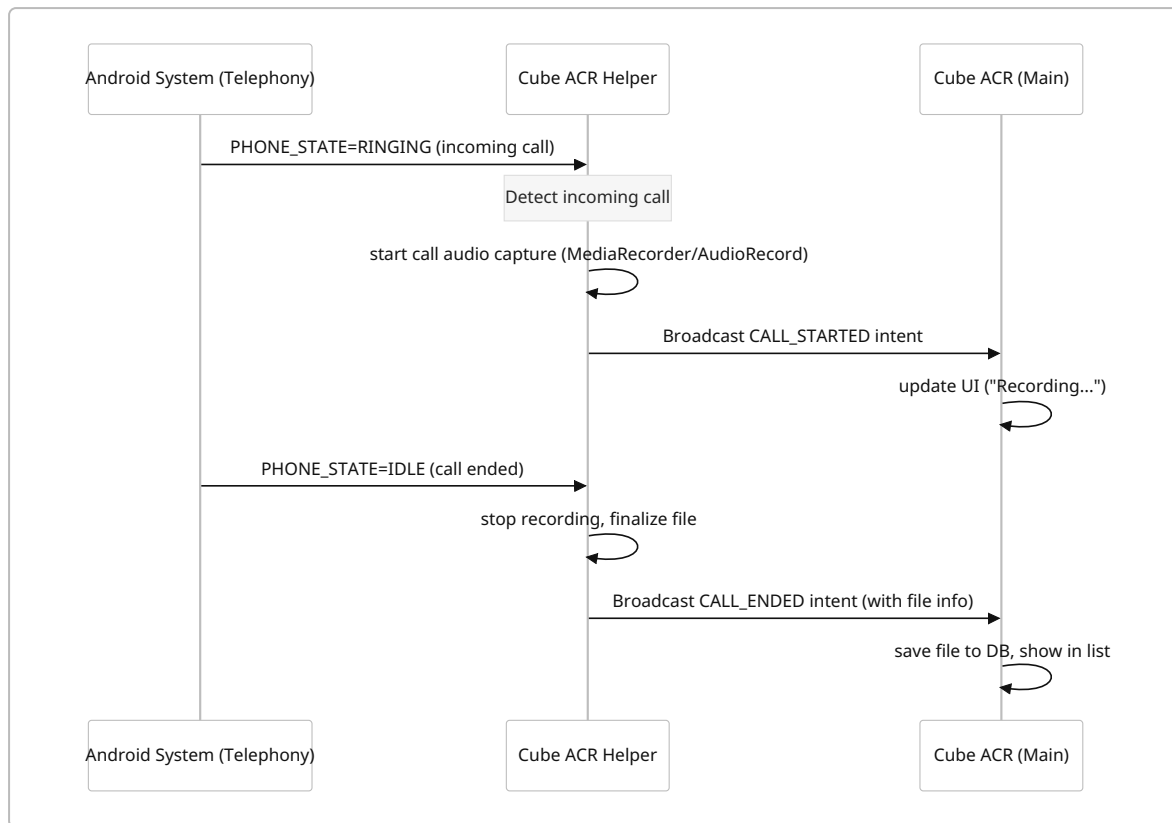
The Helper effectively hooks into the system's call audio path via accessibility features. As one developer noted: *"'Cube ACR' + 'Cube ACR Helper' ... records call audio in both directions (incoming + outgoing) natively from the system, not via the speaker. Seems to use the audio channel intended for ASR (automatic speech recognition) ... as part of the accessibility features."* ¹. In practice, the Helper uses Android's voice-

recognition or voice-call audio source (e.g. `MediaRecorder.AudioSource.VOICE_COMMUNICATION` or `VOICE_RECOGNITION`) to capture call audio.

The apps communicate via standard Android IPC. For example, after completing a recording file, the Helper may send a **broadcast Intent** or invoke a bound service method on the main app. Both apps share the same external storage (the Helper can write the file to a shared folder, and the main app reads it). There is no evidence of a ContentProvider or AIDL; instead, custom Intents and file sharing are used.

```
// Example: main app detecting if helper is installed
PackageManager pm = getPackageManager();
boolean helperInstalled;
try {
    pm.getPackageInfo("com.catalinagroup.callrecorder.helper", 0);
    helperInstalled = true;
} catch (PackageManager.NameNotFoundException e) {
    helperInstalled = false;
}
```

```
// Example: helper app sending a broadcast to main app after saving a
recording
String ACTION_COMPLETE =
    "com.catalinagroup.callrecorder.ACTION_RECORD_COMPLETE";
Intent intent = new Intent(ACTION_COMPLETE);
intent.setPackage("com.catalinagroup.callrecorder"); // target main app
intent.putExtra("file_path", recordingFilePath);
sendBroadcast(intent);
```



Android Manifest and Permissions

Each app's AndroidManifest declares its required permissions and features. Key entries include:

Cube ACR (main) manifest ² :

```

<uses-permission android:name="android.permission.RECORD_AUDIO"/>
<uses-permission android:name="android.permission.READ_PHONE_STATE"/>
<uses-permission android:name="android.permission.READ_CONTACTS"/>
<uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"/>
<uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
<uses-permission android:name="android.permission.FOREGROUND_SERVICE"/>
<uses-permission android:name="android.permission.SYSTEM_ALERT_WINDOW"/>
<uses-permission
  android:name="android.permission.REQUEST_IGNORE_BATTERY_OPTIMIZATIONS"/>
<!-- Push notifications, billing, etc. -->
<uses-permission android:name="com.google.android.c2dm.permission.RECEIVE"/>
<uses-permission android:name="com.android.vending.BILLING"/>

```

(Excerpt showing selected permissions from [28], including RECORD_AUDIO, READ_PHONE_STATE, READ_CONTACTS, SYSTEM_ALERT_WINDOW, etc.)

Cube ACR Helper manifest ⁵ ⁶ :

```

<uses-permission android:name="android.permission.RECORD_AUDIO"/>
<uses-permission android:name="android.permission.READ_PHONE_STATE"/>
<uses-permission android:name="android.permission.PROCESS_OUTGOING_CALLS"/>
<uses-permission
android:name="android.permission.BIND_ACCESSIBILITY_SERVICE"/>
<uses-permission android:name="android.permission.CAPTURE_AUDIO_OUTPUT"/>
<uses-permission android:name="android.permission.MODIFY_AUDIO_SETTINGS"/>
<uses-permission android:name="android.permission.FOREGROUND_SERVICE"/>
<uses-permission
android:name="android.permission.REQUEST_IGNORE_BATTERY_OPTIMIZATIONS"/>
<uses-permission android:name="android.permission.WAKE_LOCK"/>
<!-- Others: network, boot, storage, etc. -->

```

(Excerpt from [33]: Helper requires RECORD_AUDIO, PROCESS_OUTGOING_CALLS, BIND_ACCESSIBILITY_SERVICE, CAPTURE_AUDIO_OUTPUT (special), etc.)

A **tabulated summary** of each permission, grant type, and purpose is below (see *Permission Comparison Table*). Note: Many permissions are runtime “dangerous” (e.g. RECORD_AUDIO, READ_CONTACTS, ACCESS_FINE_LOCATION) and must be requested from the user. Others are “normal” (e.g. INTERNET) or “special” (e.g. SYSTEM_ALERT_WINDOW, BIND_ACCESSIBILITY_SERVICE).

Permission	Runtime Group	App (Main/Helper)	Purpose
RECORD_AUDIO	Dangerous (Audio)	Helper	Capture call audio via microphone/ ASR channel.
READ_PHONE_STATE	Dangerous (Phone)	Helper (+Main)	Detect call state (RINGING, OFFHOOK).
PROCESS_OUTGOING_CALLS (deprecated API)	Dangerous (Calls)	Helper	Detect outgoing calls.
READ_CALL_LOG	Dangerous (Call Log)	Helper	Identify incoming/ outgoing numbers and contact name.
READ_CONTACTS	Dangerous (Contacts)	Main	Look up caller/ callee names for display.
READ_EXTERNAL_STORAGE / WRITE_EXTERNAL_STORAGE	Dangerous (Storage)	Both	Read/write audio files on shared storage (folder for recordings).

Permission	Runtime Group	App (Main/Helper)	Purpose
<code>FOREIGN_SERVICE</code>	Normal (Process)	Both	Allow a foreground service (keeping CPU awake during recording).
<code>REQUEST_IGNORE_BATTERY_OPTIMIZATIONS</code>	Special (Whitelist)	Both	Request user to exempt app from Doze mode (important for long calls).
<code>SYSTEM_ALERT_WINDOW</code>	Special (Overlay)	Main	Show floating recording widget over call screen.
<code>BIND_ACCESSIBILITY_SERVICE</code>	Signature-only	Helper (declared)	Required to define an AccessibilityService.
<code>CAPTURE_AUDIO_OUTPUT</code>	Signature (system)	Helper	(Privileged) Capture internal audio; typically granted only to system apps. Presence suggests use of non-standard APIs or manufacturer privileges.
<code>ACCESS_NETWORK_STATE</code> , <code>INTERNET</code>	Normal	Both	Check network for cloud backups, ads, updates.
<code>BLUETOOTH</code> , <code>BLUETOOTH_CONNECT</code>	Normal (Bluetooth)	Both	(Rare) If capturing on Bluetooth calls or managing devices.
<code>ACCESS_WIFI_STATE</code>	Normal	Both	Network state (updates, optional features).
<code>WAKE_LOCK</code>	Normal	Both	Keep CPU on during recording in background.
<code>VIBRATE</code>	Normal	Both	Feedback/vibration on events.

Permission	Runtime Group	App (Main/Helper)	Purpose
<code>ACCESS_FINE_LOCATION</code> , <code>ACCESS_COARSE_LOCATION</code> , <code>ACCESS_BACKGROUND_LOCATION</code>	Dangerous (Location)	Main	Geotagging of calls feature (optional).
<code>POST_NOTIFICATIONS</code> (Android 13+)	Runtime (Notifications)	Both	Show call recording notifications in status bar (foreground service).
<code>SYSTEM_ALERT_WINDOW</code>	Special (Overlay)	Main	Floating widget during calls (user permission needed).
<code>com.google.android.gms.permission.AD_ID</code>	Normal	Both	Access Advertising ID for analytics.
<code>com.google.android.c2dm.permission.RECEIVE</code>	Normal	Main	Google Cloud Messaging (Push notifications).
<code>com.android.vending.BILLING</code>	Normal	Main	In-app purchase (premium subscription).

(Permissions table based on manifest extracts [28,33] and Android docs. “Runtime Group” indicates type of permission.)

Special app-access settings include **Accessibility** (the user must enable “Cube ACR App Connector” in Settings → Accessibility ⁷) and **Battery optimization** (the user must exempt Cube ACR & Helper from Doze for reliability) ⁸. For example, the Cube ACR FAQ advises: “make sure ... ‘Cube ACR App Connector’ is switched on in the Accessibility settings of your device.” ⁷. On Android 13+, “Allow restricted settings” is also required for the Helper (a special permission setting). The manifest also declares `<service android:permission="android.permission.BIND_ACCESSIBILITY_SERVICE">`, so the system knows the Helper includes an AccessibilityService.

Call Handling and IPC Flow

Call Event Sequence

Incoming Call: When a call rings, Android broadcasts `PHONE_STATE=RINGING`. The **Helper’s** components respond first: either an AccessibilityService callback or a BroadcastReceiver catches the event (Helper has `READ_PHONE_STATE`). The Helper then starts the recorder (e.g. `MediaRecorder` or `AudioRecord` using `AudioSource.VOICE_COMMUNICATION` or similar) to capture audio from both sides. It also notifies the main app (e.g. via a broadcast Intent) that recording has begun, so the main can update its UI (“Recording...” notification/widget).

Outgoing Call: The Helper's `PROCESS_OUTGOING_CALLS` listener (or phone API) detects when the user initiates a call. It similarly starts the recording service as soon as the call goes off-hook.

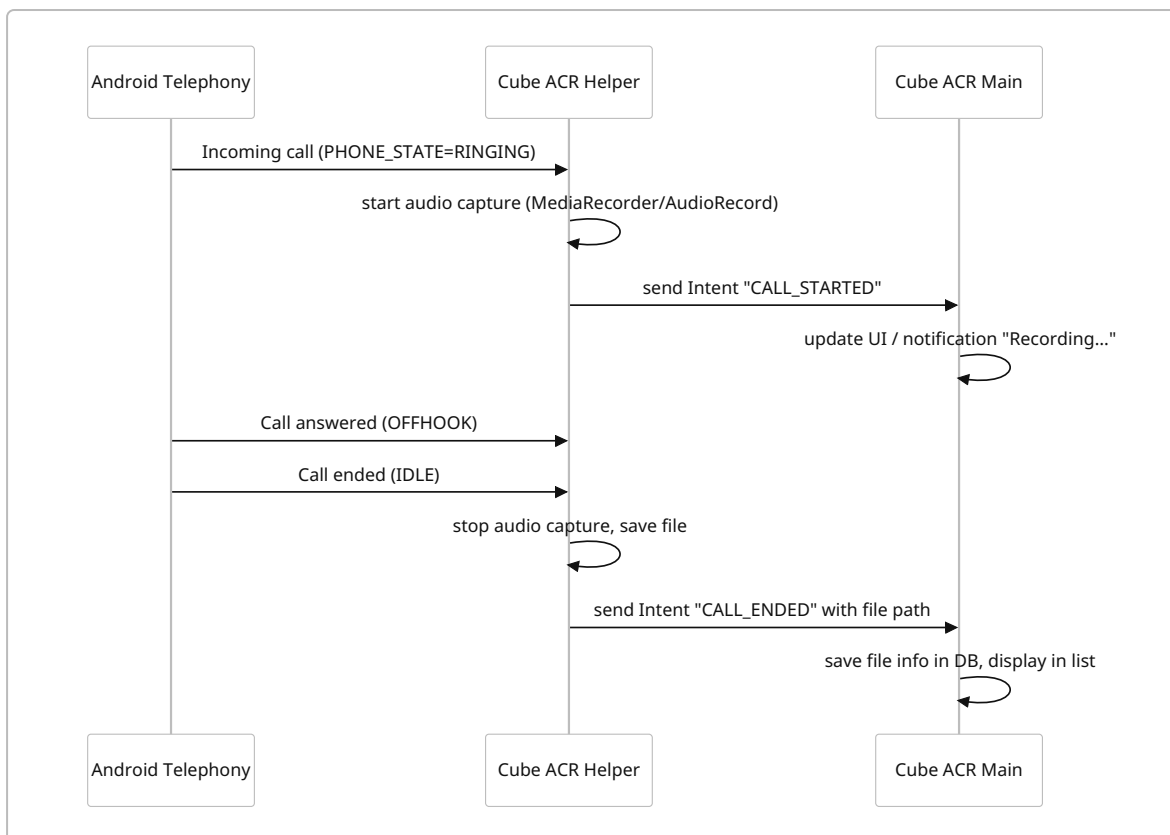
During Call: The Helper's foreground service captures audio into a temporary file (often in the app's external storage directory). Meanwhile, the main app may show a floating widget or notification (the main app holds `SYSTEM_ALERT_WINDOW` and `FOREGROUND_SERVICE` to display status).

Call End: When the call ends (`PHONE_STATE=IDLE`), the Helper stops and finalizes recording (closing the file). It then transfers control/data to the main app. Typically this is done by sending a broadcast Intent or by directly writing the file to a shared folder. For example, the Helper might send:

```
Intent done = new
Intent("com.catalinagroup.callrecorder.ACTION_RECORDING_DONE");
done.setPackage("com.catalinagroup.callrecorder"); // ensure only Main app
receives it
done.putExtra("file_path", recordingFile.getAbsolutePath());
context.sendBroadcast(done);
```

The **Main app** has a registered `BroadcastReceiver` for this action. Upon receiving it, the main app reads `file_path`, adds the file to its database (assigning caller info via Contacts lookup), and refreshes the recordings list in the UI.

A *mermaid* sequence diagram illustrates this flow:



(Figure: Simplified call-event flow. The Helper handles phone events and audio capture; the Main handles UI and file storage.)

The Helper thus “bridges” the telephony subsystem and the main app, allowing the main UI to remain responsive while the heavy recording work is done in the background. On Android 9+, this approach (using accessibility and alternative audio channels) is essentially required, because direct capture of in-call audio was restricted by Google’s policy ⁹.

IPC Mechanisms

The apps use standard Android IPC:

- **Broadcast Intents:** For one-way signals. As shown above, the Helper can broadcast custom actions (e.g. `CALL_STARTED`, `CALL_ENDED`) to the main app. The main app must declare `<receiver>` entries for these and possibly require a specific permission token (`DYNAMIC_RECEIVER_NOT_EXPORTED_PERMISSION`) to prevent other apps from spoofing them. Similarly, the main app can send Intents to the Helper (e.g. user-initiated commands from the UI).
- **Bound Service (optional):** It is possible the Helper exposes a bound service (with a custom AIDL or simple IBinder) so the main app could bind and call methods. However, in practice most call-recording apps use broadcasts or `ContentResolver` URIs. If a service is used, it would be declared with `android:exported="false"` and guarded by a signature-level permission.
- **Shared Files:** Both apps have external storage permission. The Helper may write recordings to a public directory (e.g. `/sdcard/CubeAcr/`). The main app simply scans that folder (or is told the path via Intent) and then moves/manages the file. No special file-sharing IPC is needed beyond normal file I/O, since the files are world-readable (on older Android) or both have requested storage access.

Example IPC code snippet: Main app registering a receiver for the helper’s broadcast:

```
IntentFilter filter = new
IntentFilter("com.catalinagroup.callrecorder.ACTION_RECORD_COMPLETE");
filter.setPriority(IntentFilter.SYSTEM_HIGH_PRIORITY);
registerReceiver(new BroadcastReceiver() {
    @Override public void onReceive(Context ctx, Intent intent) {
        String path = intent.getStringExtra("file_path");
        // Process new recording (update DB/UI)
    }
}, filter);
```

Runtime Permissions and Special Access

At install time, both apps declare many permissions, but most **dangerous** ones must be granted at runtime. For example, when first launched:

- **Audio and Phone:** The user must allow `RECORD_AUDIO` and `READ_PHONE_STATE` (often prompted together). This is required by the Helper to capture audio and detect calls. If denied, recording will fail.
- **Storage:** Both apps ask for `READ/WRITE_EXTERNAL_STORAGE` (on Android ≤ 12) or use the Storage Access Framework (Android 13+). Denying this prevents saving recordings.
- **Contacts:** The main app may request `READ_CONTACTS` if caller ID display is enabled. Denying it disables name tagging.
- **Overlay:** The main app's UI might have a floating widget. This requires the **"Display over other apps"** (`SYSTEM_ALERT_WINDOW`) permission, which is a special user-granted access in Settings (Android will prompt the user to allow it, not a runtime dialog).
- **Battery Optimization:** Both apps request the user to exclude them from battery optimizations (via `REQUEST_IGNORE_BATTERY_OPTIMIZATIONS`). This must be granted in system settings, otherwise Android may kill the recording service in background.

Importantly, the **Accessibility** setting is not a runtime permission but a system toggle. Users must manually enable "Cube ACR App Connector" in Settings → Accessibility ⁷. In this mode, the Helper's AccessibilityService can monitor the call UI and system logs. Banking apps often detect this: as one user reported, HSBC blocked access, warning "Detected apps...pose a potential security risk: Cube ACR Helper. To restore access...disable accessibility or uninstall." ¹⁰. In other words, enabling the Helper's accessibility triggers security flags in some apps (esp. finance apps), reflecting the privacy concern of recording.

Call Recording Mechanics

Under the hood, the Helper uses Android's audio API to capture call audio. On modern Android (10+), direct capture of the `VOICE_CALL` audio stream is blocked. Cube ACR's strategy (as observed) is to use the **voice-recognition** channel. Code-wise, it likely uses something like:

```
MediaRecorder recorder = new MediaRecorder();
recorder.setAudioSource(MediaRecorder.AudioSource.VOICE_COMMUNICATION); // or
VOICE_RECOGNITION
recorder.setOutputFormat(MediaRecorder.OutputFormat.MPEG_4);
recorder.setAudioEncoder(MediaRecorder.AudioEncoder.AAC);
recorder.setOutputFile(filePath);
recorder.prepare();
recorder.start();
```

This captures both uplink and downlink via the speech recognition audio path, effectively recording in-call voice. Android docs note these audio sources route through the digital signal processor used for speech, which only works when AccessibilityService is active ¹. The helper may also try `AudioPlaybackCapture` (API29+) with a suitable configuration, but the blog suggests using the "ASR channel".

During recording, the Helper holds a **foreground service** so Android keeps the audio stream open and the app alive. The manifest declares:

```
<uses-permission android:name="android.permission.FOREGROUND_SERVICE"/>
```

and the helper's service is started with `startForeground()` (displaying a persistent notification).

When stopping, `recorder.stop()` is called and the file is finalized. The Helper may temporarily write to its own private cache and then move/rename the file into the main app's recordings directory.

Storage and Data Flow

By default, recordings are stored in a folder like `/sdcard/Cube ACR/` or `/storage/emulated/0/CallRecordings/`. The exact path can be set in the app's settings. Both apps have declared storage permissions, allowing them to read/write here.

Data flow: 1. Helper writes raw audio to file on storage. 2. Helper notifies Main of new file (via Intent or by file observer). 3. Main reads the file path, moves it into its own managed directory (if needed), and adds metadata (timestamp, duration, contact). 4. Main may upload to cloud if the user enabled backup.

Files are not shared via ContentProvider; they are accessed by normal file I/O. That is simpler but requires careful permission management.

Lifecycle & Foreground Handling

- **Helper:** Runs an AccessibilityService (its lifecycle is controlled by user enabling/disabling in Settings). It also likely has a background Service. During idle periods it might be dormant or only monitoring call state. On incoming call, it may spin up (start) its recording service. The recording service must run in foreground to avoid being killed (especially on modern Android). Hence the `FOREGROUND_SERVICE` permission and notification.
- **Main:** Typically idle when not in use. It might start a persistent "CallRecorderService" when a call begins to keep the process alive until recording ends (using `startForeground` to avoid Doze). The manifest grants `WAKE_LOCK` so CPU stays awake during a long recording. When the call (and recording) ends, the service can stop.

If battery optimizations are **not** disabled, Android may kill the Helper or Main while recording, causing failures. The app prompts the user to whitelist it (common in call recording apps).

Device and Vendor Workarounds

Call recording behavior varies by manufacturer:

- On **OnePlus, Xiaomi, etc.**, there may be built-in call recording engines. Cube ACR does not use these; instead it uses the accessibility/ASR trick. Some devices may have special patches (e.g. OnePlus enabling internal audio capture).

- Bluetooth/Headset calls: Cube ACR FAQ notes that recording on a Bluetooth headset often does **not** work with this approach ¹¹. The ASR channel hack can only capture the phone mic and speaker mix, not audio from an external device.
- Android version differences: On Android 6–9, it suggests using `voice_call (software)` source; on 10–14 use `voice_recognition (software)` ¹². These settings correspond to how the Helper configures the audio capture source based on OS version.
- VoIP apps (WhatsApp, Viber, etc.): The Helper monitors the foreground app via accessibility and can trigger manual recording if a known VoIP app is in use. But due to Android security, not all VoIP streams are capturable. The FAQ admits some devices cannot record VoIP ¹³.

In summary, Cube ACR has built-in workarounds per-device (likely hidden in code or preset profiles) and gives user tips (auto-speaker mode, different audio source) for unsupported models ¹⁴ ¹⁵.

Play Store Policy and Helper Distribution

Google Play's policy restricts call recording. To comply, Cube moved the Helper off Google Play to other stores and direct APK download ¹⁶. The main app still lives on Play, but the Helper is offered on Galaxy Store, AppGallery, Amazon, and via their website (see *Download Helper .APK* link) ¹⁶. This sideloading approach allows calling the necessary accessibility APIs without violating Play rules (Google disallows call-recording apps, but allows a separate “app connector” with generic description).

Thus, installation typically goes: 1. Install **Cube ACR** from Play. 2. It prompts for **Cube ACR Helper**; user installs it (from outside Play). 3. User grants permissions to Helper (Accessibility, Microphone, etc.). 4. Link between them is established.

If the main app is uninstalled, the Helper **alone** would not function well: it has no UI or file manager. In practice, without the main app, recordings may be lost or not shown. The FAQ explicitly warns: “*Cube ACR Helper is NOT a substitute for Cube ACR. It's the utility app which works in tandem with Cube ACR, to make recording possible and smooth.*” ¹⁷.

Detection by Other Apps

Because Cube ACR uses Accessibility, **security-sensitive apps** (banking, corporate VPNs, etc.) often refuse to run when Cube's services are active ¹⁰. As one user saw, the HSBC bank app blocked access and listed “Cube ACR Helper” as a risk. The advice is to disable accessibility when using those apps. This is a known side-effect: Android accessibility gives broad privileges (screen reading, etc.), so banks treat any active accessibility service as a threat. Cube ACR itself provides no hidden malicious behavior, but the mere presence of its helper triggers these warnings.

Implementation Snippets

Below are illustrative code snippets (simplified) for critical functions:

1. **Detecting Helper App:** The main app checks if the helper package is installed:

```
private boolean isHelperInstalled(Context ctx) {
    PackageManager pm = ctx.getPackageManager();
    try {
        pm.getPackageInfo("com.catalinagroup.callrecorder.helper", 0);
        return true;
    } catch (PackageManager.NameNotFoundException e) {
        return false;
    }
}
```

1. **Starting Recording (Helper):** The helper's background service might use `MediaRecorder`:

```
MediaRecorder recorder = new MediaRecorder();
recorder.setAudioSource(MediaRecorder.AudioSource.VOICE_COMMUNICATION);
recorder.setOutputFormat(MediaRecorder.OutputFormat.MPEG_4);
recorder.setAudioEncoder(MediaRecorder.AudioEncoder.AAC);
recorder.setOutputFile(file.getAbsolutePath());
recorder.prepare();
recorder.start();
// ... on call end ...
recorder.stop();
recorder.release();
```

1. **Using AudioRecord (alternative):**

```
int sampleRate = 44100;
int bufferSize = AudioRecord.getMinBufferSize(sampleRate,
        AudioFormat.CHANNEL_IN_MONO,
        AudioFormat.ENCODING_PCM_16BIT);
AudioRecord audioRecord = new AudioRecord(
    MediaRecorder.AudioSource.VOICE_COMMUNICATION,
    sampleRate, AudioFormat.CHANNEL_IN_MONO,
    AudioFormat.ENCODING_PCM_16BIT, bufferSize);
audioRecord.startRecording();
// Read from audioRecord into buffer and write to file...
audioRecord.stop();
audioRecord.release();
```

1. **IPC via Service Binding (example):** The helper could expose a local service:

```
// In Helper: define a Binder
public class HelperService extends Service {
    private final LocalBinder binder = new LocalBinder();
    public class LocalBinder extends Binder {
        CubeAcrHelperService getService() { return
CubeAcrHelperService.this; }
    }
    @Override public IBinder onBind(Intent intent) { return binder; }
```

```

    public void startRecording() { /* ... */ }
    public void stopRecording() { /* ... */ }
}

```

The main app could bind and call:

```

ServiceConnection conn = new ServiceConnection() {
    public void onServiceConnected(ComponentName c, IBinder binder) {
        CubeAcrHelperService service = ((LocalBinder)binder).getService();
        service.startRecording();
    }
    ...
};
Intent intent = new
Intent().setClassName("com.catalinagroup.callrecorder.helper",
    "com.catalinagroup.callrecorder.helper.HelperService");
bindService(intent, conn, Context.BIND_AUTO_CREATE);

```

Best Practices and Security Notes

- **Separation of Concerns:** Splitting into two apps minimizes permissions granted to the UI app. The Helper can have broad access, while the main app remains relatively lightweight. This is advisable for any app needing “system-level” access (similar to how some DRM or modem-assisted apps use helpers).
- **Least Privilege:** Both apps should only request needed permissions. For example, the main app should not hold `PROCESS_OUTGOING_CALLS` (it doesn’t) and should minimize use of Accessibility APIs. The helper, being headless, can request the heavy ones.
- **Secure Storage:** Recordings may contain private conversations. Cube ACR allows PIN-locking the app, but by default files are just on external storage. A best practice would be to encrypt recordings or store them in app-private storage (Android 11+ scoped storage) to protect privacy.
- **Transparency:** The user should be clearly informed (as Cube ACR does in its UI/FAQ) that recording uses accessibility, that law compliance is the user’s responsibility, and that accessibility can affect other apps.
- **Anti-detection:** Because banking apps block accessibility services, an advanced implementation might detect when sensitive apps are launched and temporarily pause recording or disable the service (though this is tricky and not user-friendly). At minimum, the app should warn the user as the bank app did.

Sources

- Cube ACR official site and FAQ ⁷ ⁴
- Cube ACR Play Store listing (mentions accessibility usage) ¹⁸
- APK manifest data from Cube ACR and Helper (from APKMirror analysis) ² ¹⁹

- Developer/user posts (Reddit, blogs) on how Cube ACR works ¹ ¹⁰
 - Android documentation (MediaRecorder, permissions, AccessibilityService) ¹ (developer blog) and AndroidX refs.
-

¹ Alex Radzishevsky digital home

<https://radzishevsky.com/blog/>

² Call Recorder - Cube ACR 2.4.269 (120-640dpi) (Android 5.0+) APK Download by Cube Apps Limited - APKMirror

<https://www.apkmirror.com/apk/cube-systems/call-recorder-cube-acr/call-recorder-cube-acr-2-4-269-release/call-recorder-cube-acr-2-4-269-android-apk-download/>

³ ⁴ ⁵ ⁶ ⁹ ¹⁷ ¹⁹ Cube ACR Helper 1.0.17 APK Download by Cube Apps Limited - APKMirror

<https://www.apkmirror.com/apk/cube-systems/cube-acr-helper/cube-acr-helper-1-0-17-release/cube-acr-helper-1-0-17-android-apk-download/>

⁷ ⁸ ¹¹ ¹² ¹³ ¹⁴ ¹⁵ ¹⁶ FAQ | Cube Call Recorder ACR | Cube

<https://cubeacr.app/faq.html>

¹⁰ Alternates for Cube ACR? : r/androidapps

https://www.reddit.com/r/androidapps/comments/1ebvcos/alternates_for_cube_acr/

¹⁸ Call Recorder - Cube ACR - Apps on Google Play

https://play.google.com/store/apps/details?id=com.catalinagroup.callrecorder&hl=en_US